

**LOSTLINK – A CLOUD-BASED PLATFORM FOR
REPORTING AND MANAGING
LOST & FOUND ITEMS**

CLOUD COMPUTING

Submitted by

**Ragul A
Rahul P
Rakesh R**

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

**COMPUTER SCIENCE AND
ENGINEERING**



**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

RAJALAKSHMI ENGINEERING COLLEGE

May 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled “**LOSTLINK – A CLOUD-BASED PLATFORM FOR REPORTING AND MANAGING LOST & FOUND ITEMS**” is the bonafide work of **RAGUL A (230701252), RAHUL P (230701254), RAKESH R (230701257)**, who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Ragul A

Rahul P

Rakesh R

Department of Computer Science
and Engineering

Rajalakshmi Engineering College

SIGNATURE

Dr.M.Ayyadurai

Supervisor

Associate Professor

Department of Computer Science
and Engineering

Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on_____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to , Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. M. AYYADURAI M.E.,Ph.D.**, Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

ABSTRACT

The Lost and Found Management System is a cloud-based web application designed to provide an efficient, secure, and user-friendly platform for managing lost and found items within organizations, institutions, campuses, or public communities. Traditional lost-and-found systems often rely on manual record maintenance, physical registers, and delayed communication, which can lead to inefficiency, data loss, and reduced chances of item recovery. This project aims to overcome these limitations by digitizing the entire process through an intelligent and centralized management system. The application enables users to report lost or found items by submitting detailed information such as item name, category, description, location, date, and images. The system stores and manages this information using Azure Cosmos DB, which provides scalable and high-performance cloud database support. Item images uploaded by users are securely stored using Azure Blob Storage, ensuring efficient media handling and accessibility. The frontend of the application is developed using React.js, providing a responsive and interactive user interface, while the backend is built using Node.js and Express.js to handle API requests, authentication, business logic, and database communication. One of the major features of the system is its intelligent matching mechanism, which automatically compares lost and found item reports based on multiple parameters such as item characteristics, category, location, and descriptive keywords. This significantly improves the chances of identifying potential matches and reduces the manual effort required by administrators. The project also integrates AI-based functionalities using the Gemini API to enhance item description analysis and improve user interaction through smarter processing capabilities. To provide a seamless and real-time experience, the application incorporates Socket.IO technology for instant notifications and live updates whenever matching items are detected or status changes occur. The system also includes secure authentication and authorization mechanisms to protect user data and restrict unauthorized access. Middleware-based validation and protected routes are implemented to maintain application security and ensure reliable request handling. The proposed system is designed with scalability, maintainability, and cloud integration in mind, making it suitable for deployment in real-world environments. The responsive design allows users to access the platform across desktops and mobile devices with ease. By automating the lost-and-found workflow and leveraging modern cloud technologies, the system minimizes manual management efforts, improves operational efficiency, and increases the likelihood of successful item recovery. Overall, the Lost and Found Management System serves as a smart and practical digital solution that combines modern web technologies, cloud computing, real-time communication, and intelligent matching features to create a reliable platform for managing lost and found items effectively.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	Introduction	1
1.1	Problem Statement	2
1.2	Objective of the Project	4
1.3	Scope and Boundaries	6
1.4	Stakeholders and End Users	8
1.5	Technologies Used	10
1.6	Organization of the Report	12
2	System Design and Architecture	14
2.1	Requirement Summary (Functional & Non-Functional)	15
2.2	Proposed Solution Overview	17
2.3	Cloud Deployment Strategy	19
2.4	Infrastructure Requirements	21
2.5	Azure Services Mapping and Justification	23
3	DevOps Implementation	26
3.1	Continuous Integration and Deployment (CI/CD) Setup	27
3.2	Terraform Infrastructure-as-Code (IaC)	29
3.3	Containerization Strategy (Docker)	31
3.4	Kubernetes Orchestration (AKS Deployment, Scaling)	33
3.5	GenAI Integration and Azure AI Service Mapping	35
4	Cloud Operations and Security	38
4.1	DevSecOps Integration (CodeQL / SonarCloud / ZAP Scans)	39
4.2	Monitoring and Observability (Azure Monitor / App Insights)	41
4.3	Access Control (RBAC Overview)	43

4.4	Blue–Green Deployment & Disaster Recovery Planning	45
5	Results and Discussion	48
5.1	Implementation Summary	49
5.2	Challenges Faced and Resolutions	51
5.3	Performance or Cost Observations	53
5.4	Key Learnings and Team Contributions	55
6	Conclusion and Future Enhancement	58
6.1	Conclusion	59
6.2	Future Scope	61
	References	63
	Appendices	65

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Managing lost and found items using traditional manual methods is inefficient and unreliable. Most institutions maintain physical registers or depend on verbal communication and notice boards to report and recover lost belongings. These approaches often lead to delayed communication, missing records, duplicate reports, lack of transparency, and reduced chances of item recovery. Additionally, there is no centralized platform that allows users to easily report lost or found items, upload images, track item status, or receive instant notifications about possible matches. Manual searching and comparison of item details consume significant time and effort for both users and administrators. Therefore, there is a need for a smart, secure, and cloud-based Lost and Found Management System that automates the process of reporting, storing, matching, and recovering lost items while providing real-time communication and improved user accessibility.

1.2 OBJECTIVE OF THE PROJECT

The main objective of the Lost and Found Management System is to develop a secure, scalable, and cloud-based web application that simplifies and automates the process of managing lost and found items. The system is designed to provide an efficient and user-friendly platform where users can report lost or found belongings, upload detailed item information along with images, and search for matching records. The project aims to reduce the limitations of traditional manual methods by introducing a centralized digital solution that improves communication and increases the chances of successful item recovery. The application incorporates an intelligent matching mechanism that analyzes item details to identify possible matches between lost and found reports. Real-time notifications and updates are implemented using Socket.IO technology to ensure instant communication between users and administrators. Azure Cosmos DB is used for secure and scalable storage of user and item information, while Azure Blob Storage is utilized for efficient image management. The system also integrates AI-based functionalities through the Gemini API to improve item description analysis and enhance user interaction. Overall, the project focuses on reducing manual effort, improving operational efficiency, ensuring data security, and providing accessibility across multiple devices through a responsive and modern web interface.

1.3 SCOPE AND BOUNDARIES

The main objective of the Lost and Found Management System is to develop a secure, scalable, and cloud-based web application that simplifies and automates the process of managing lost and found items. The system is designed to provide an efficient and user-friendly platform where users can report lost or found belongings, upload detailed item information along with images, and search for matching records. The project aims to reduce the limitations of traditional manual methods by introducing a centralized digital solution that improves communication and increases the chances of successful item recovery. The application incorporates an intelligent matching mechanism that analyzes item details to identify possible matches between lost and found reports. Real-time notifications and updates are implemented using Socket.IO technology to ensure instant communication between users and administrators. Azure Cosmos DB is used for secure and scalable storage of user and item information, while Azure Blob Storage is utilized for efficient image management. The system also integrates AI-based functionalities through the Gemini API to improve item description analysis and enhance user interaction. Overall, the project focuses on reducing manual effort, improving operational efficiency, ensuring data security, and providing accessibility across multiple devices through a responsive and modern web interface.

1.4 STAKEHOLDERS AND END USERS

Stakeholders:

1. **Project Developers / Team Members** - Responsible for designing, developing, testing, and maintaining the Lost and Found Management System, including modules such as user authentication, item reporting, intelligent matching, real-time notifications, cloud storage integration, and AI-assisted functionalities.
2. **Administrators** - Monitor and manage the platform by verifying reports, handling disputes, tracking item recovery status, and ensuring smooth and secure operation of the system.
3. **Organization / Institution Management** - Institutions, colleges, offices, or communities that deploy and maintain the platform to provide an organized and efficient lost and found management solution for their members.
4. **Cloud Service Providers** - Support the application infrastructure through services such as Azure Cosmos DB, Azure Blob Storage, and cloud hosting required for secure data storage and scalability.

End Users:

1. **Students in Educational Institutions** - who use the platform to report lost belongings, search for found items, and receive real-time updates regarding item recovery.
2. **Employees in Offices or Organizations** - who utilize the system to manage misplaced personal or official belongings within workplace environments.
3. **Residents or Visitors in Communities and Public Spaces** - who interact with the platform to report and recover lost items efficiently through a centralized digital system.
4. **Event Participants and Administrative Staff** - who use the application during events or institutional activities to track, manage, and coordinate lost and found item recovery processes.

1.5 TECHNOLOGIES USED

The Lost and Found Management System is developed using modern web development and cloud technologies to ensure performance, scalability, and reliability.

Technology	Purpose
React.js	Frontend user interface development
Node.js	Backend runtime environment
Express.js	Server-side API development
Azure Cosmos DB	Cloud database for storing user and item data
Azure Blob Storage	Storage for uploaded item images
Socket.IO	Real-time communication and notifications
Gemini API	AI-based processing and intelligent functionalities
JavaScript	Core programming language
HTML & CSS	Frontend structure and styling
REST APIs	Communication between frontend and backend
JWT Authentication	Secure user authentication and authorization
Git & GitHub	Version control and project management

1.6 ORGANIZATION OF THE REPORT

The project report is organized into multiple chapters to provide a structured understanding of the system development process.

Chapter 1 – Introduction:

Provides an overview of the project, problem statement, objectives, scope, stakeholders, technologies used, and report organization.

Chapter 2 – Literature Survey:

Discusses existing systems, related research works, and comparative analysis.

Chapter 3 – System Analysis and Design:

Explains system architecture, requirements analysis, database design, UML diagrams, and workflow design.

Chapter 4 – Implementation:

Describes frontend and backend development, API integration, cloud services integration, and core functionalities.

Chapter 5 – Testing and Results:

Covers testing methodologies, test cases, output screenshots, and performance evaluation.

Chapter 6 – Conclusion and Future Enhancements:

Summarizes the project outcomes, achievements, limitations, and possible future improvements.

References:

Contains the list of books, websites, research papers, and resources referred to during project development.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

The Lost and Found Management System is designed to provide a secure, efficient, and cloud-based platform for managing lost and found items within institutions and communities. The system requirements are categorized into functional and non-functional requirements to ensure proper system performance, usability, security, and scalability.

Functional Requirements

The functional requirements describe the core operations and features that the system must perform. The application should allow users to register and log in securely using authentication mechanisms. Users must be able to create lost or found item reports by entering item details such as name, category, description, location, date, and uploading related images. The system should provide functionalities for searching, filtering, and viewing item records based on different criteria. An intelligent matching mechanism should compare lost and found reports and identify potential matches automatically. Real-time notifications must be provided to users whenever a possible match or status update occurs. Administrators should have the ability to monitor reports, manage users, verify records, and handle disputes when necessary. The system should also support cloud-based image storage and secure database management for handling large amounts of user and item data efficiently.

Non-Functional Requirements

The non-functional requirements define the quality attributes and operational standards of the system. The application should provide high availability, scalability, and reliability through cloud-based deployment. The user interface must be responsive and accessible across desktops and mobile devices. The system should ensure data security through authentication, authorization, and secure API communication. Performance optimization is required to support fast data retrieval and efficient image handling. The platform should maintain real-time communication capabilities with minimal latency. The application should also be maintainable, modular, and easily extendable for future enhancements such as advanced AI-based image recognition or analytics integration.

2.2 PROPOSED SOLUTION OVERVIEWS

The proposed solution is a cloud-based Lost and Found Management System developed to automate and simplify the process of reporting, managing, and recovering lost items. The system provides a centralized web platform where users can submit lost or found item reports along with detailed descriptions and images. Unlike traditional manual methods, the proposed solution offers intelligent item matching, real-time notifications, secure cloud storage, and responsive accessibility through modern web technologies. The frontend of the application is developed using React.js, which provides an interactive and responsive user interface for seamless user experience. The backend is implemented using Node.js and Express.js to manage API requests, authentication, business logic, and communication with cloud services. Azure Cosmos DB is used as the primary cloud database for securely storing user information and item records, while Azure Blob Storage is utilized for storing uploaded item images efficiently. The system integrates Socket.IO technology to enable real-time communication and instant notifications between users and administrators. Additionally, AI-based functionalities are incorporated using the Gemini API to improve item description analysis and enhance matching accuracy. The intelligent matching module compares item details such as category, description, location, and date to identify potential matches between lost and found reports. Overall, the proposed solution aims to reduce manual effort, improve operational efficiency, increase recovery success rates, and provide a scalable and secure digital platform for managing lost and found activities in real-world environments.

2.3 CLOUD DEPLOYMENT STRATEGY

The Lost and Found Management System follows a cloud-based deployment strategy to ensure scalability, availability, security, and efficient resource management. The application is deployed using Microsoft Azure cloud services, which provide reliable infrastructure and seamless integration for storage, database management, and application hosting. The frontend application is deployed as a web application service, allowing users to access the platform through modern web browsers on multiple devices. The backend server is hosted in a cloud environment using Node.js runtime support to manage APIs, authentication, business logic, and real-time communication. Azure Cosmos DB is deployed as the primary NoSQL cloud database to handle structured and semi-structured data related to users, lost items, found items, and notifications. Azure Blob Storage is used for securely storing uploaded images and media files associated with item reports. The deployment architecture is designed to support scalability by allowing cloud resources to handle increasing numbers of users and data dynamically. Real-time communication using Socket.IO is integrated into the deployment environment to ensure low-latency notifications and live updates. Security measures such as environment variables, protected APIs, secure authentication mechanisms, and controlled access permissions are implemented to protect sensitive user and application data. The cloud deployment strategy ensures high availability, fault tolerance, data persistence, and simplified maintenance, making the system suitable for deployment in educational institutions, organizations, and public communities.

2.4 INFRASTRUCTURE REQUIREMENTS

The infrastructure requirements for the Lost and Found Management System include both hardware and software components necessary for development, deployment, and system operation.

Hardware Requirements

- Processor: Intel Core i5 or higher
- RAM: Minimum 8 GB
- Storage: Minimum 256 GB SSD
- Internet Connection: Stable broadband connection for cloud communication and deployment
- Client Devices: Desktop, laptop, tablet, or smartphone with modern browser support

Software Requirements

- Operating System: Windows, Linux, or macOS
- Frontend Technologies: React.js, HTML, CSS, JavaScript
- Backend Technologies: Node.js, Express.js
- Database: Azure Cosmos DB
- Cloud Storage: Azure Blob Storage
- Real-Time Communication: Socket.IO
- AI Integration: Gemini API
- Development Tools: Visual Studio Code, Git, GitHub
- Package Manager: npm
- Web Browser: Google Chrome, Microsoft Edge, Firefox, or Safari

Network and Cloud Requirements

- Microsoft Azure cloud account for hosting and storage services
- Secure API communication over HTTPS
- Internet connectivity for real-time synchronization and cloud access
- Scalable cloud infrastructure for handling multiple concurrent users and uploaded media.

These infrastructure components collectively support the smooth functioning, scalability, and reliability of the application.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

The Lost and Found Management System utilizes multiple Microsoft Azure services to provide secure, scalable, and efficient cloud-based functionality. Each Azure service is selected based on the specific requirements of the application.

Azure Service	Purpose	Justification
Azure Cosmos DB	Stores user data, item records, notifications, and report details	Provides scalable, high-performance, and globally distributed NoSQL database support with fast data access
Azure Blob Storage	Stores uploaded images and media files	Efficiently handles large volumes of image storage with secure cloud access and high availability
Azure App Service / Cloud Hosting	Hosts frontend and backend applications	Enables scalable, reliable, and cloud-based deployment with simplified maintenance
Azure Authentication & Security Services	Supports secure access and protected communication	Ensures secure authentication, authorization, and API protection
Azure Networking Services	Manages secure communication between services	Provides secure and reliable cloud connectivity for application components

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STATERGY

The Lost and Found Management System follows a Continuous Integration and Continuous Deployment (CI/CD) approach to automate the processes of building, testing, and deploying the application. The CI/CD pipeline helps improve development efficiency, reduce manual deployment errors, and ensure faster delivery of application updates. The source code for both frontend and backend modules is managed using Git and GitHub for version control and collaboration. Whenever changes are pushed to the repository, automated workflows are triggered to validate the code, install dependencies, build the application, and execute testing procedures. The CI/CD process ensures that newly added features or bug fixes do not negatively affect the stability of the existing system. The deployment pipeline is configured to automatically deploy the frontend and backend services to the Azure cloud environment after successful validation and testing. Environment variables and sensitive credentials are securely managed using cloud configuration settings. Automated deployment reduces downtime and enables consistent application updates across development, testing, and production environments. The CI/CD setup improves software quality, supports agile development practices, and enables efficient maintenance and scalability of the Lost and Found Management System.

Figure 1. Github Actions Workflow

The pipeline follows 2-stage deployment pattern:

1. **Backend:** Deploys the main application (as a Container App).
2. **Frontend:** Builds and deploys the static web app (React).

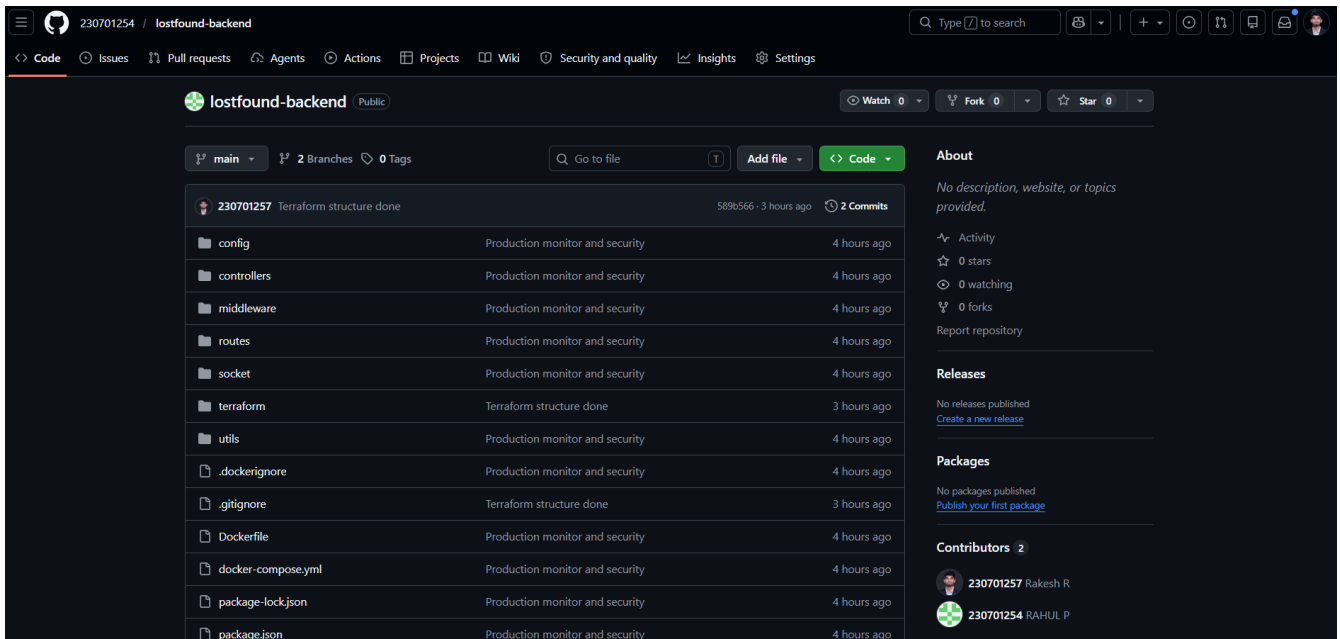


Figure 2. Backend Deployment Job Run

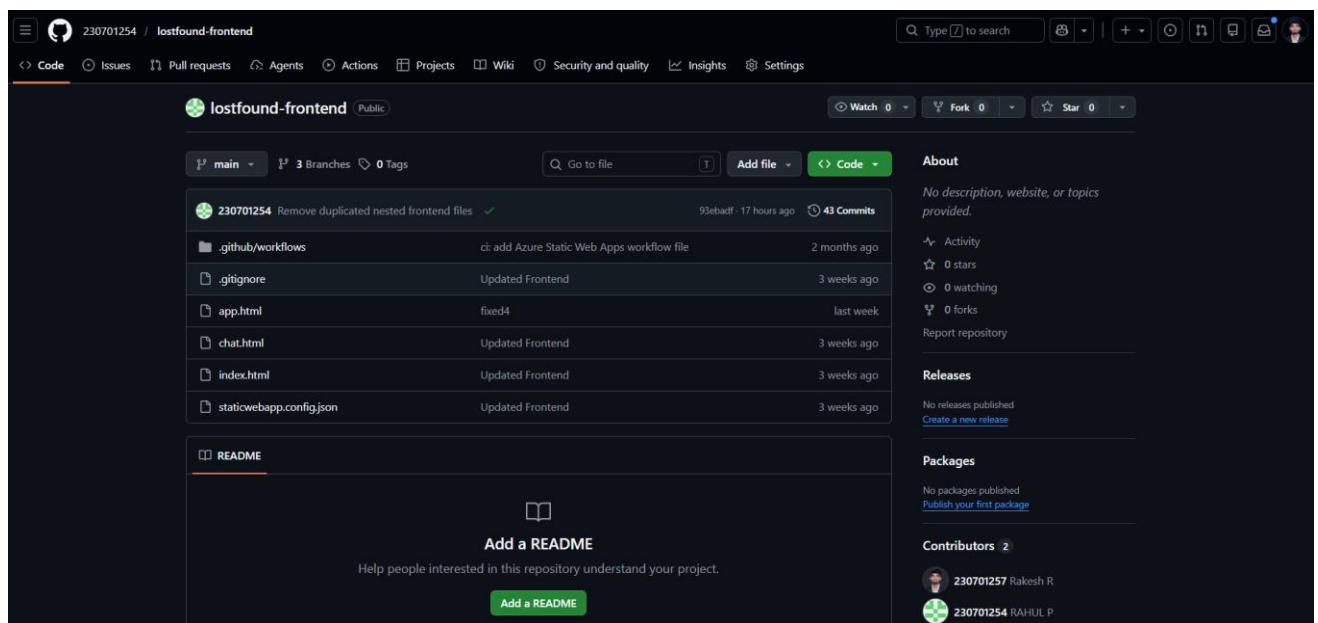


Figure 3. Frontend Deployment Job Run

Required GitHub Secrets

Table 7. List of Github Secrets

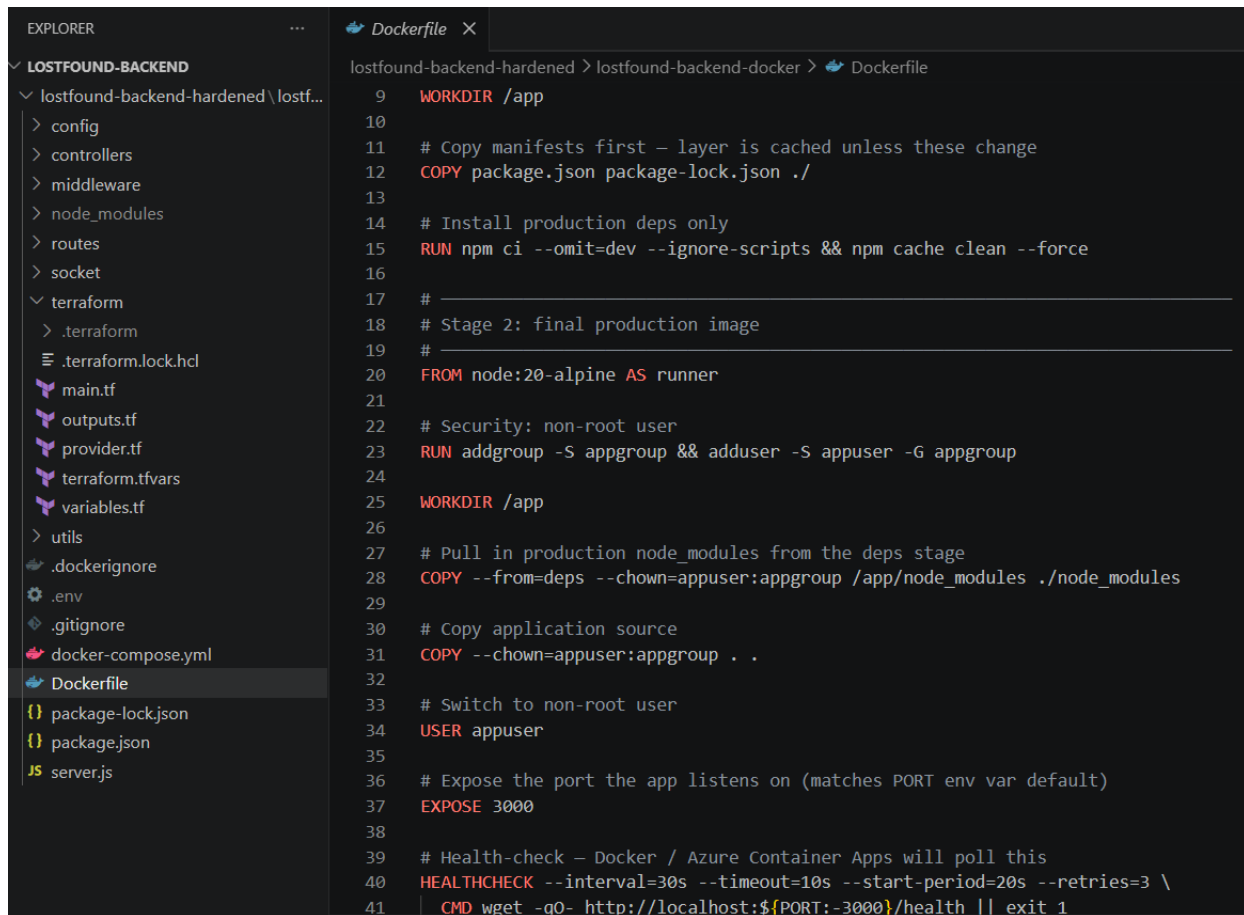
1	AZURE_CLIENT_ID
2	AZURE_CLIENT_SECRET
3	AZURE_STATIC_WEB_APPS_API_TOKEN
4	AZURE_STORAGE_ACCOUNT_NAME
5	AZURE_SUBSCRIPTION_ID
6	AZURE_TENANT_ID
7	CONTENT_SAFETY_KEY
8	DB_PASSWORD
9	DOCKERHUB_TOKEN
10	DOCKERHUB_USERNAME
11	GEMINI_API_KEY

3.2 TERRAFORM INFRASTRUCTURE-AS-CODE (IaC)

The Lost and Found Management System adopts Infrastructure-as-Code (IaC) principles using Terraform to automate and manage cloud infrastructure resources efficiently. Terraform enables infrastructure provisioning through declarative configuration files, reducing the need for manual cloud setup and configuration. Using Terraform, Azure resources such as Azure Cosmos DB, Azure Blob Storage, application hosting services, networking configurations, and access management settings can be provisioned and managed consistently across different environments. Infrastructure configurations are stored as code, allowing version control, repeatability, and easier collaboration among development teams. Terraform improves infrastructure reliability by ensuring that the deployment environment remains consistent during development, testing, and production stages. It also simplifies scaling and maintenance by allowing infrastructure changes to be applied systematically using reusable configuration templates. The use of Infrastructure-as-Code enhances automation, minimizes configuration errors, supports disaster recovery, and improves deployment efficiency for the overall system architecture.

3.3 CONTAINERIZATION STRATEGY (DOCKER)

The Lost and Found Management System utilizes Docker-based containerization to package and deploy application components in a consistent and isolated environment. Docker containers encapsulate the frontend application, backend services, dependencies, and runtime configurations, ensuring portability and compatibility across different systems. Separate Docker containers are created for the React.js frontend and Node.js backend services. Docker images are built using Dockerfiles that define the application environment, dependencies, execution commands, and configuration settings. This approach eliminates environment inconsistencies and simplifies application deployment across development, testing, and production environments. Containerization improves scalability, resource utilization, and deployment efficiency while reducing conflicts caused by system dependencies. Docker also enables faster deployment, easier rollback mechanisms, and simplified application maintenance. The use of containerized services supports modern DevOps practices and prepares the system for orchestration and cloud-native deployment. Overall, Docker provides a lightweight, portable, and efficient deployment solution for the Lost and Found Management System.



The image shows a code editor interface with a file explorer on the left and a code editor on the right. The file explorer, titled 'EXPLORER', shows a project structure for 'LOSTFOUND-BACKEND'. The 'lostfound-backend-hardened' directory is expanded, showing subdirectories like 'config', 'controllers', 'middleware', 'node_modules', 'routes', 'socket', 'terraform', and 'utils'. The 'terraform' directory is further expanded, showing files like '.terraform', '.terraform.lock.hcl', 'main.tf', 'outputs.tf', 'provider.tf', 'terraform.tfvars', and 'variables.tf'. The 'Dockerfile' is selected and highlighted. The code editor on the right shows the content of the 'Dockerfile' with line numbers 9 through 41. The Dockerfile contains instructions for building a Docker image, including setting the working directory, copying manifests, installing dependencies, creating a non-root user, and exposing a port for health checks.

```
9  WORKDIR /app
10
11 # Copy manifests first – layer is cached unless these change
12 COPY package.json package-lock.json ./
13
14 # Install production deps only
15 RUN npm ci --omit=dev --ignore-scripts && npm cache clean --force
16
17 # -----
18 # Stage 2: final production image
19 # -----
20 FROM node:20-alpine AS runner
21
22 # Security: non-root user
23 RUN addgroup -S appgroup && adduser -S appuser -G appgroup
24
25 WORKDIR /app
26
27 # Pull in production node_modules from the deps stage
28 COPY --from=deps --chown=appuser:appgroup /app/node_modules ./node_modules
29
30 # Copy application source
31 COPY --chown=appuser:appgroup . .
32
33 # Switch to non-root user
34 USER appuser
35
36 # Expose the port the app listens on (matches PORT env var default)
37 EXPOSE 3000
38
39 # Health-check – Docker / Azure Container Apps will poll this
40 HEALTHCHECK --interval=30s --timeout=10s --start-period=20s --retries=3 \
41 | CMD wget -qO- http://localhost:${PORT:-3000}/health || exit 1
```

Figure 4. Docker file

3.4 KUBERNETES ORCHESTRATION

To support scalable and reliable deployment, the Lost and Found Management System can be orchestrated using Kubernetes through Azure Kubernetes Service (AKS). Kubernetes manages containerized applications by automating deployment, scaling, networking, and service management across cloud infrastructure. The frontend and backend Docker containers are deployed as Kubernetes pods within the AKS cluster. Kubernetes deployment configurations define replica sets, service exposure, resource allocation, and container health monitoring. Load balancing mechanisms distribute incoming traffic efficiently across running instances to maintain system performance during high user activity. AKS provides automatic scaling capabilities that dynamically increase or decrease resources based on application demand. This ensures efficient resource utilization and high availability of services. Kubernetes also supports self-healing mechanisms that automatically restart failed containers and maintain application stability. Using Kubernetes orchestration improves fault tolerance, deployment reliability, scalability, and operational efficiency. The AKS-based deployment architecture makes the system suitable for handling large-scale usage scenarios while maintaining consistent performance and availability.

3.5 GEN-AI INTEGRATION AND AZURE AI SERVICE MAPPING

The Lost and Found Management System integrates Generative AI functionalities to enhance user interaction and improve the efficiency of item matching and description analysis. AI-based processing is implemented using the Gemini API and cloud-based AI services to provide intelligent assistance within the platform. The GenAI integration supports automated analysis of item descriptions, keyword extraction, enhanced text processing, and improved matching suggestions between lost and found reports. By analyzing descriptive text provided by users, the AI module helps identify meaningful patterns and increases the probability of detecting related items more accurately. Azure AI-related services can also be mapped to support future enhancements such as natural language processing, image recognition, chatbot assistance, and predictive analytics. These services can help automate administrative tasks, improve search capabilities, and provide intelligent recommendations to users.

AI Service	Purpose	Benefit
Gemini API	AI-based description analysis and intelligent processing	Improves item matching accuracy and user interaction
Azure Cognitive Services	Natural language and image analysis	Enables advanced AI-based automation and recognition features
Azure AI APIs	Intelligent search and analytics	Enhances search performance and decision-making capabilities

The integration of GenAI technologies improves automation, user experience, and intelligent processing capabilities, making the system more efficient and adaptable for future AI-driven enhancements

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

Cloud operations and security play a critical role in ensuring the reliability, scalability, performance, and protection of the Lost and Found Management System. Since the application is deployed using cloud-based infrastructure and services, it is essential to implement operational monitoring, secure access management, automated security testing, and continuous system maintenance. The system utilizes Microsoft Azure cloud services along with modern DevOps and DevSecOps practices to maintain secure and efficient application operations. The cloud infrastructure is designed to support scalable deployment, real-time communication, secure data storage, and high availability of services. Azure cloud services such as Azure Cosmos DB and Azure Blob Storage are integrated to manage application data and uploaded images securely. Monitoring and observability tools are used to track system performance, detect failures, and improve troubleshooting efficiency. Security mechanisms such as Role-Based Access Control (RBAC), authentication, and protected APIs are implemented to prevent unauthorized access and protect sensitive user information. Additionally, DevSecOps practices are integrated into the software development lifecycle to automate security analysis and vulnerability detection during development and deployment processes. Automated security scanning tools such as CodeQL, SonarCloud, and OWASP ZAP help identify code vulnerabilities, insecure configurations, and potential security threats before deployment. Real-time monitoring services such as Azure Monitor and Azure Application Insights provide insights into system health, resource utilization, and application performance. Overall, cloud operations and security mechanisms ensure that the Lost and Found Management System remains scalable, secure, reliable, and capable of handling real-world deployment requirements efficiently.

4.1 DevSecOps Integration (CodeQL / SonarCloud / ZAP Scans)

The Lost and Found Management System incorporates DevSecOps practices to integrate security into every phase of the software development lifecycle. Security testing and vulnerability analysis are continuously performed during development, testing, and deployment to improve application security and reduce risks. Static code analysis tools such as CodeQL and SonarCloud are used to scan the source code for vulnerabilities, insecure coding practices, maintainability issues, and potential security flaws. These tools help identify problems such as injection vulnerabilities, authentication weaknesses, insecure dependencies, and coding errors before deployment. Dynamic Application Security Testing (DAST) is performed using OWASP ZAP (Zed Attack Proxy), which analyzes the running application to detect real-world vulnerabilities such as Cross-Site Scripting (XSS), insecure APIs, session management issues, and misconfigured security settings. Security scanning is integrated into the CI/CD pipeline to automate validation and ensure secure deployments. The DevSecOps integration improves software quality, enhances security compliance, and minimizes the possibility of security breaches in the production environment.

4.2 Monitoring and Observability (Azure Monitor / Application Insights)

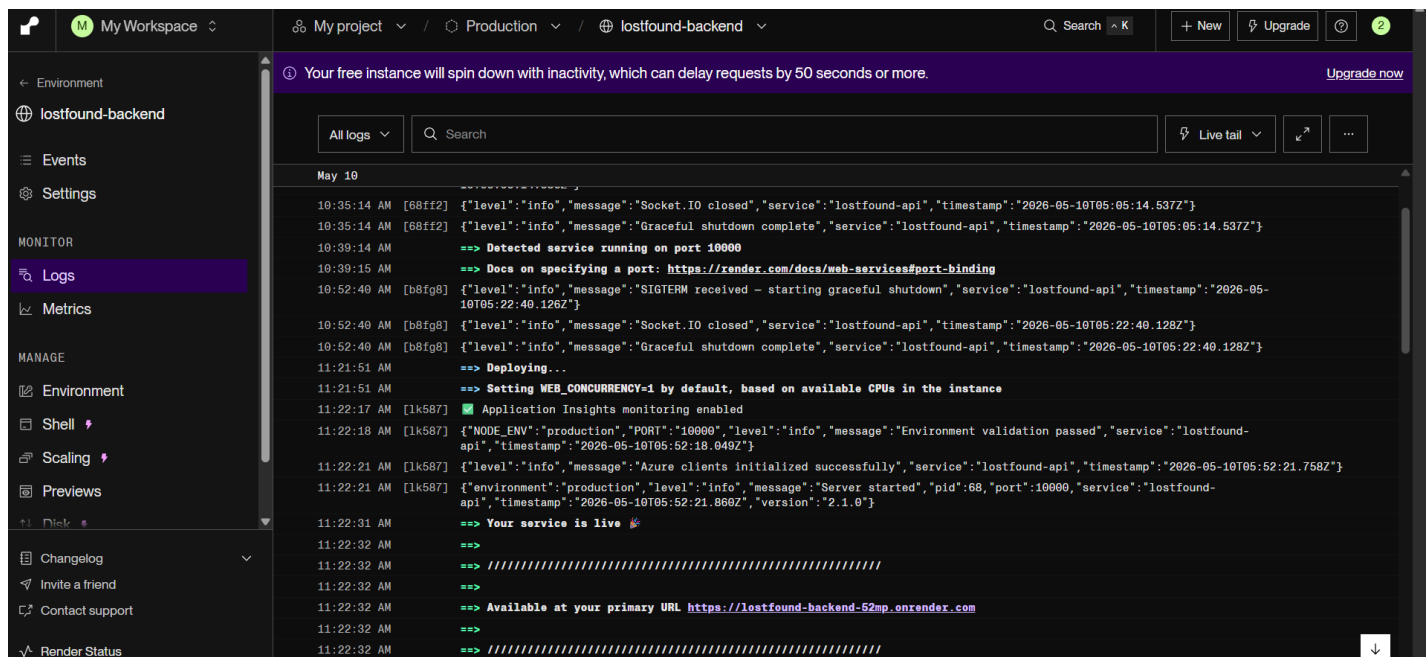
The Lost and Found Management System utilizes monitoring and observability services to maintain system performance, reliability, and operational efficiency. Azure Monitor is used to monitor cloud infrastructure resources, application availability, network performance, and service health. It helps administrators track resource utilization, identify abnormal activities, and receive alerts when system issues occur.

Azure Application Insights is integrated into the application to collect telemetry data such as API response times, request failures, application logs, error reports, and user activity analytics. It enables developers to monitor frontend and backend performance in real time and quickly diagnose issues affecting the system. Distributed tracing and dependency tracking features help analyze communication between application services and cloud resources.

These monitoring and observability mechanisms improve system reliability, simplify troubleshooting, reduce downtime, and support proactive maintenance of the Lost and Found Management System.

4.3 Access Control (RBAC Overview)

The Lost and Found Management System implements Role-Based Access Control (RBAC) to ensure secure and controlled access to system resources and functionalities. RBAC restricts user actions based on predefined roles and permissions, helping prevent unauthorized access and protecting sensitive data. The system primarily defines roles such as regular users and administrators. Regular users can register, log in, report lost or found items, upload images, search records, and receive notifications. Administrators are granted additional permissions to monitor reports, manage users, verify item details, resolve disputes, and oversee platform operations. Authentication is implemented using secure token-based mechanisms such as JSON Web Tokens (JWT), ensuring secure session management and protected API access. Middleware-based authorization checks validate user permissions before allowing access to protected functionalities. Cloud-level access permissions are also configured to secure Azure resources such as databases and storage services. The RBAC implementation enhances system security, improves data protection, ensures proper access management, and supports secure cloud operations within the Lost and Found Management System.



The screenshot displays the Render Logs interface for a project named 'lostfound-backend'. The interface includes a sidebar with navigation options: Environment, Settings, MONITOR (with Logs selected), Metrics, and MANAGE (with Environment, Shell, Scaling, and Previews). The main area shows a log stream for May 10. The logs include timestamps, log levels, and messages. Key messages include: 'Socket.IO closed', 'Graceful shutdown complete', 'Detected service running on port 10000', 'SIGTERM received - starting graceful shutdown', 'Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance', 'Application Insights monitoring enabled', 'Environment validation passed', 'Azure clients initialized successfully', 'Server started', and 'Your service is live'. The logs also mention the primary URL: <https://lostfound-backend-52mp.onrender.com>. A notification at the top states: 'Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.' with an 'Upgrade now' button.

```
May 10
10:35:14 AM [68ff2] {"level":"info","message":"Socket.IO closed","service":"lostfound-api","timestamp":"2026-05-10T05:05:14.537Z"}
10:35:14 AM [68ff2] {"level":"info","message":"Graceful shutdown complete","service":"lostfound-api","timestamp":"2026-05-10T05:05:14.537Z"}
10:39:14 AM
10:39:15 AM ==> Detected service running on port 10000
==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
10:52:40 AM [b8fg8] {"level":"info","message":"SIGTERM received - starting graceful shutdown","service":"lostfound-api","timestamp":"2026-05-10T05:22:40.126Z"}
10:52:40 AM [b8fg8] {"level":"info","message":"Socket.IO closed","service":"lostfound-api","timestamp":"2026-05-10T05:22:40.128Z"}
10:52:40 AM [b8fg8] {"level":"info","message":"Graceful shutdown complete","service":"lostfound-api","timestamp":"2026-05-10T05:22:40.128Z"}
11:21:51 AM
11:21:51 AM ==> Deploying...
11:21:51 AM ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
11:22:17 AM [1k587] Application Insights monitoring enabled
11:22:18 AM [1k587] {"NODE_ENV":"production","PORT":"10000","level":"info","message":"Environment validation passed","service":"lostfound-api","timestamp":"2026-05-10T05:52:18.049Z"}
11:22:21 AM [1k587] {"level":"info","message":"Azure clients initialized successfully","service":"lostfound-api","timestamp":"2026-05-10T05:52:21.758Z"}
11:22:21 AM [1k587] {"environment":"production","level":"info","message":"Server started","pid":68,"port":10000,"service":"lostfound-api","timestamp":"2026-05-10T05:52:21.860Z","version":"2.1.0"}
11:22:31 AM ==> Your service is live 🎉
11:22:32 AM ==>
11:22:32 AM ==>
11:22:32 AM ==> Available at your primary URL https://lostfound-backend-52mp.onrender.com
11:22:32 AM ==>
11:22:32 AM ==>
11:22:32 AM ==>
11:22:32 AM ==>
```

Figure 5. Render Logs

CHAPTER 5 - RESULTS AND DISCUSSION

The Results and Discussion section presents the overall outcomes achieved during the development and deployment of the Lost and Found Management System. The project successfully demonstrates the implementation of a cloud-based web application capable of managing lost and found activities efficiently through modern technologies and cloud infrastructure. The developed system provides functionalities such as user authentication, item reporting, image upload, intelligent matching, real-time notifications, and secure cloud storage. The integration of Azure cloud services enabled scalable and reliable storage management for user data and uploaded images. Real-time communication using Socket.IO improved user interaction by providing instant notifications and status updates. AI-assisted functionalities enhanced item description analysis and contributed to improved matching efficiency between lost and found reports. The responsive frontend design ensured accessibility across different devices, including desktops and mobile platforms. The project also demonstrated the practical implementation of DevOps, DevSecOps, cloud deployment, monitoring, and security practices. Overall, the system achieved its primary objective of automating and simplifying the lost and found management process while improving operational efficiency and user experience.

5.1 IMPLEMENTATION SUMMARY

The Lost and Found Management System was successfully implemented using a modern full-stack web development approach combined with cloud-based infrastructure and services. The frontend of the application was developed using React.js to provide a responsive and interactive user interface. The backend was implemented using Node.js and Express.js to manage APIs, authentication, business logic, and communication with cloud resources. Azure Cosmos DB was integrated as the primary cloud database for storing user information, item records, and application data securely. Azure Blob Storage was used for efficient storage and retrieval of uploaded item images. Socket.IO technology was implemented to provide real-time communication and notifications between users and administrators. AI-assisted functionalities using the Gemini API were incorporated to enhance item description analysis and improve matching accuracy. The application was deployed using cloud infrastructure and containerization practices to ensure scalability and maintainability. Monitoring, security, and access control mechanisms were also integrated to support reliable system operation. The implemented system successfully fulfilled the functional and non-functional requirements defined during the project planning phase.

5.2 Challenges Faced and Resolutions

During the development of the Lost and Found Management System, several technical and operational challenges were encountered. One of the major challenges involved integrating multiple cloud services such as Azure Cosmos DB, Azure Blob Storage, and real-time communication services within a single application environment. Proper configuration and secure communication between these services required careful management of credentials, APIs, and cloud resources. Another challenge was implementing real-time notifications using Socket.IO while maintaining stable communication between frontend and backend services. This issue was resolved through proper event handling, optimized socket connections, and backend synchronization techniques. Managing image uploads and cloud storage integration also required handling file validation, storage permissions, and efficient retrieval mechanisms. The intelligent matching process posed challenges related to handling inconsistent or incomplete user-provided item descriptions. To improve matching accuracy, AI-assisted description analysis and keyword-based comparison techniques were integrated into the system. Security concerns such as API protection, authentication, and secure cloud access were addressed using JWT-based authentication, middleware validation, and role-based access control mechanisms. Performance optimization and deployment configuration also required troubleshooting during containerization and cloud deployment stages. These challenges were resolved through testing, monitoring, configuration adjustments, and deployment automation practices.

5.3 Performance or Cost Observations

The performance of the Lost and Found Management System was evaluated based on application responsiveness, database operations, real-time communication efficiency, and cloud resource utilization. The use of React.js provided fast and responsive frontend rendering, while Node.js and Express.js enabled efficient backend request handling. Azure Cosmos DB demonstrated high-speed data retrieval and scalable database performance for handling user and item records. Real-time notifications implemented using Socket.IO provided low-latency communication between users and administrators, improving overall user experience. Azure Blob Storage efficiently managed uploaded images and reduced storage-related performance bottlenecks. Monitoring tools such as Azure Monitor and Application Insights helped identify resource usage patterns and optimize system performance. From a cost perspective, cloud resource consumption depended mainly on database operations, storage usage, network traffic, and application hosting requirements. Azure's scalable pricing model allowed efficient resource management by allocating services based on application demand. Containerization and cloud deployment practices also contributed to optimized infrastructure utilization and simplified maintenance. Overall, the system demonstrated satisfactory performance, scalability, and manageable operational costs suitable for small- to medium-scale deployment environments.

5.4 Key Learnings and Team Contributions

The development of the Lost and Found Management System provided valuable technical and practical learning experiences for the project team. Team members gained hands-on experience in full-stack web development, cloud computing, real-time communication, database management, and AI integration. The project enhanced understanding of frontend technologies such as React.js and backend technologies including Node.js and Express.js. The team also learned practical implementation of cloud services such as Azure Cosmos DB, Azure Blob Storage, and cloud deployment strategies. Experience in DevOps and DevSecOps practices, including CI/CD pipelines, security scanning, containerization using Docker, and Kubernetes orchestration, contributed significantly to understanding modern software engineering workflows. Real-time communication using Socket.IO improved knowledge of event-driven programming and live synchronization techniques. The integration of AI-based functionalities using the Gemini API provided insights into Generative AI applications and intelligent data processing. Team collaboration, version control using Git and GitHub, problem-solving, debugging, testing, and deployment management also strengthened project management and teamwork skills. Each team member contributed to different aspects of the project such as frontend development, backend implementation, database integration, cloud deployment, testing, documentation, and security management. These collaborative efforts ensured the successful completion and deployment of the system.

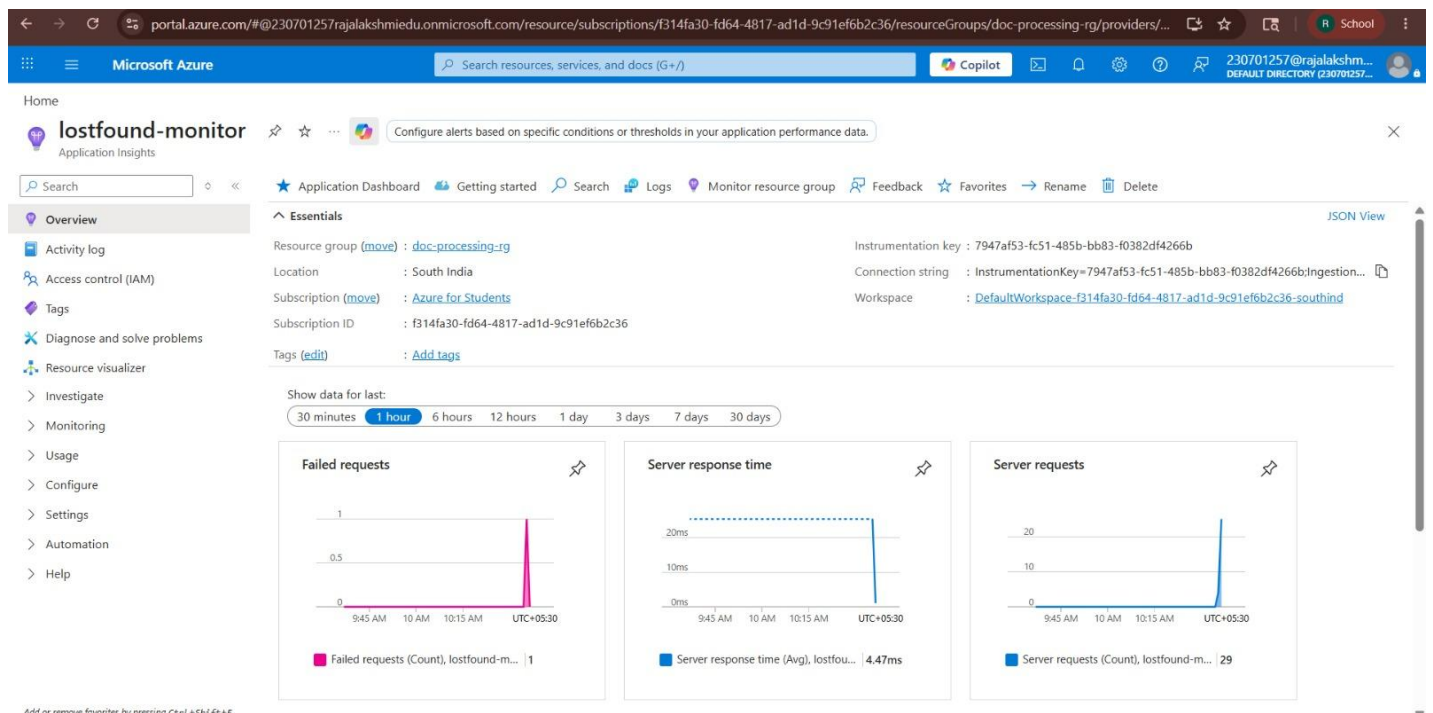


Figure 6. Azure monitor

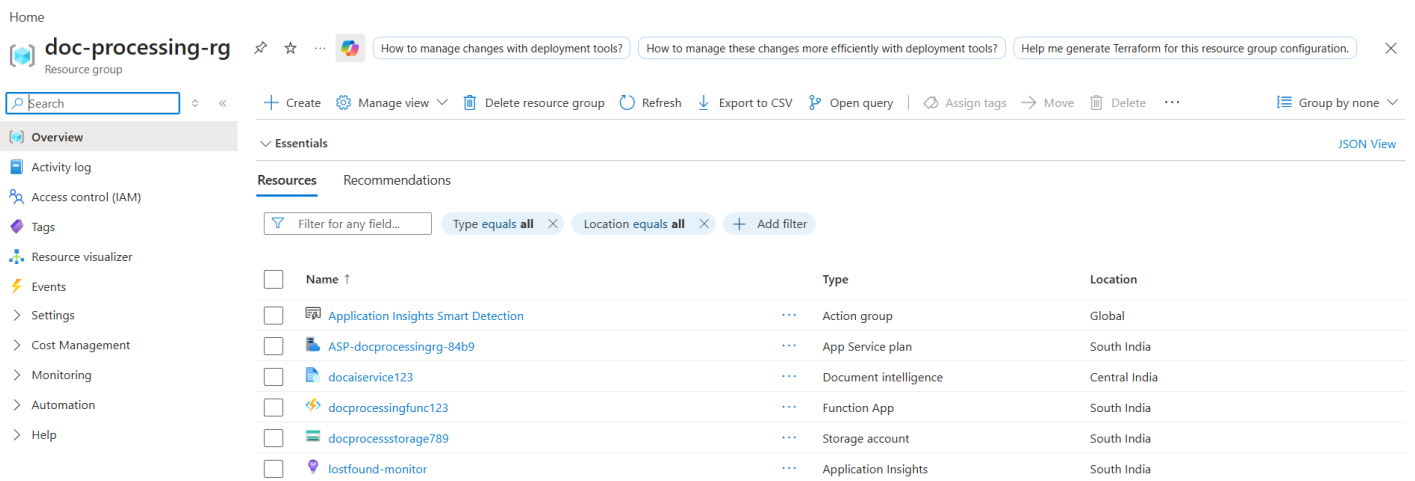


Figure 7. Azure Services

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

The Lost and Found Management System successfully demonstrates the implementation of a secure, scalable, and cloud-based platform for managing lost and found activities efficiently. By integrating modern web technologies, cloud services, real-time communication, and AI-assisted functionalities, the system addresses the limitations of traditional manual lost-and-found processes and provides an organized digital solution. The project achieved its objectives by enabling users to report lost or found items, upload images, receive real-time notifications, and identify possible matches through intelligent processing mechanisms. Cloud integration improved scalability, reliability, and accessibility, while security measures ensured safe handling of user and application data. The system serves as a practical example of modern full-stack application development integrated with cloud computing, DevOps, DevSecOps, and AI technologies. It provides a strong foundation for future enhancements and large-scale deployment in real-world environments.

6.1 Conclusion

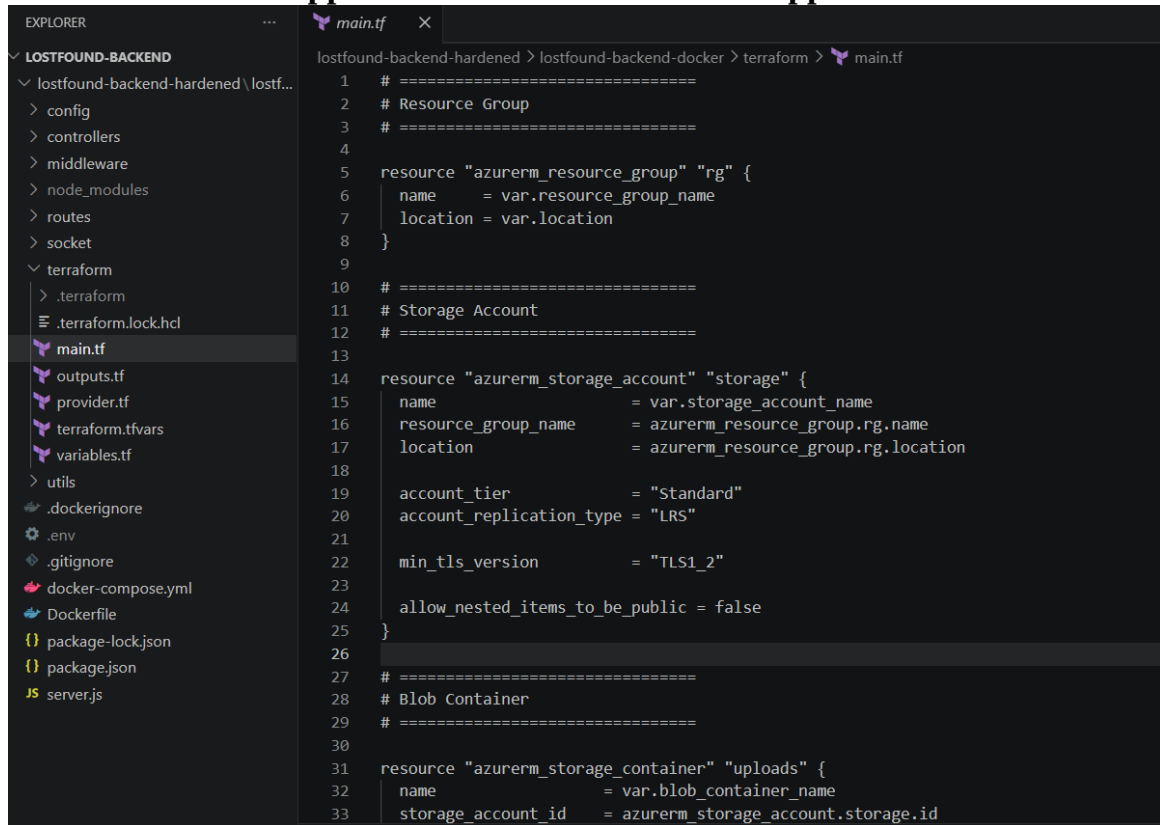
The Lost and Found Management System was developed to provide an efficient and automated solution for managing lost and found items within institutions and communities. The system successfully replaced traditional manual methods with a centralized digital platform that improves communication, record management, and item recovery efficiency. The integration of React.js, Node.js, Express.js, Azure Cosmos DB, Azure Blob Storage, Socket.IO, and AI-assisted functionalities enabled the development of a scalable, responsive, and secure cloud-based application. Real-time notifications, intelligent matching mechanisms, secure authentication, and cloud deployment significantly improved user experience and operational effectiveness. The project also demonstrated the practical implementation of cloud operations, DevOps practices, security integration, monitoring systems, and containerized deployment strategies. Overall, the developed system provides a reliable and modern solution for lost and found management while supporting scalability, maintainability, and future technological enhancements.

6.2 Future Scope

Although the Lost and Found Management System successfully implements the core functionalities required for efficient item management, several future enhancements can further improve the system's capabilities and performance. Future developments may include advanced AI-powered image recognition models for automatic item identification and similarity detection using computer vision techniques. Integration of chatbot-based support systems can improve user assistance and automate common queries. Mobile application development for Android and iOS platforms can provide better accessibility and user convenience. Additional security features such as multi-factor authentication, biometric verification, and advanced fraud detection mechanisms can further strengthen platform security. The system can also be enhanced with analytics dashboards, predictive reporting, and recommendation systems to improve administrative decision-making. Future cloud optimizations may include serverless architectures, microservices-based deployment, advanced Kubernetes orchestration, and automated scaling strategies for handling large-scale deployments. Integration with institutional ID systems and GPS-based location tracking can also improve item verification and recovery processes. These future enhancements can transform the system into a more intelligent, scalable, and fully automated platform suitable for deployment across larger organizations and public environments.

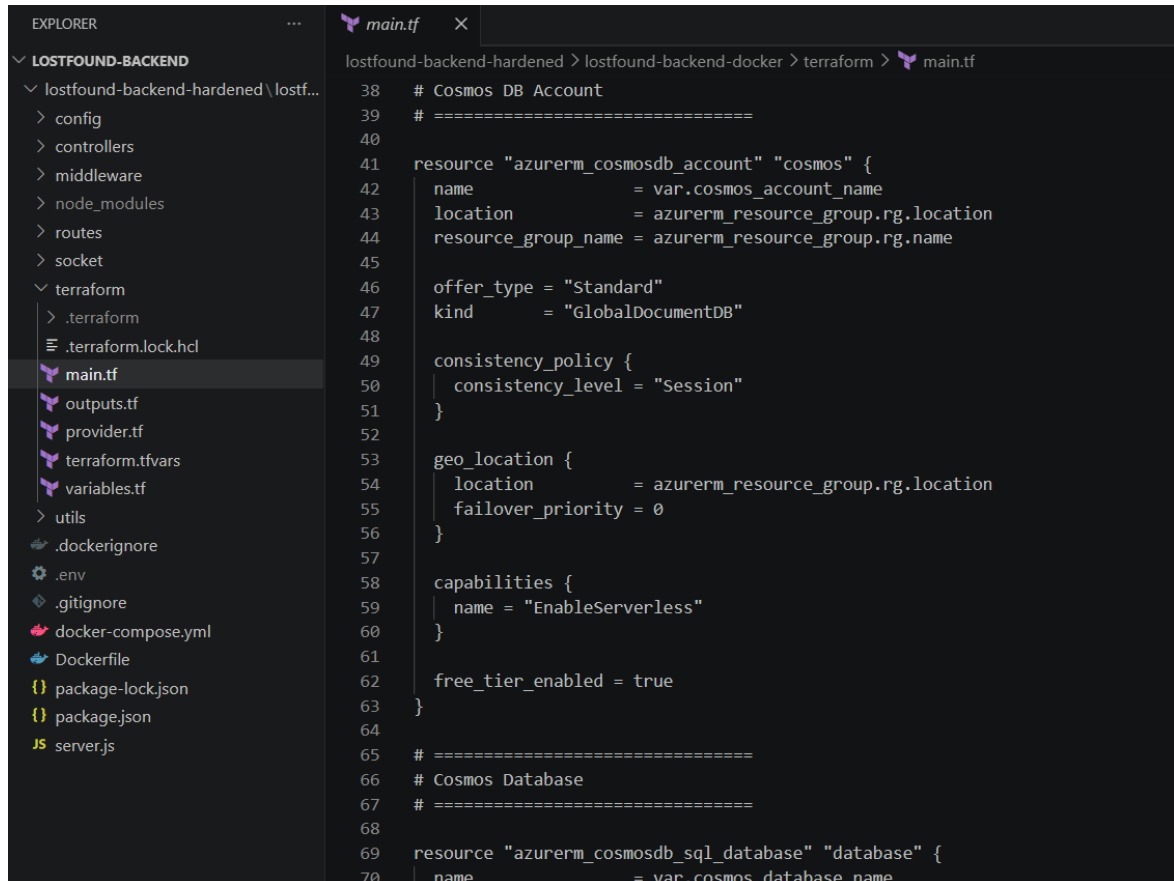
APPENDICES

Appendix A – Terraform Code Snippets



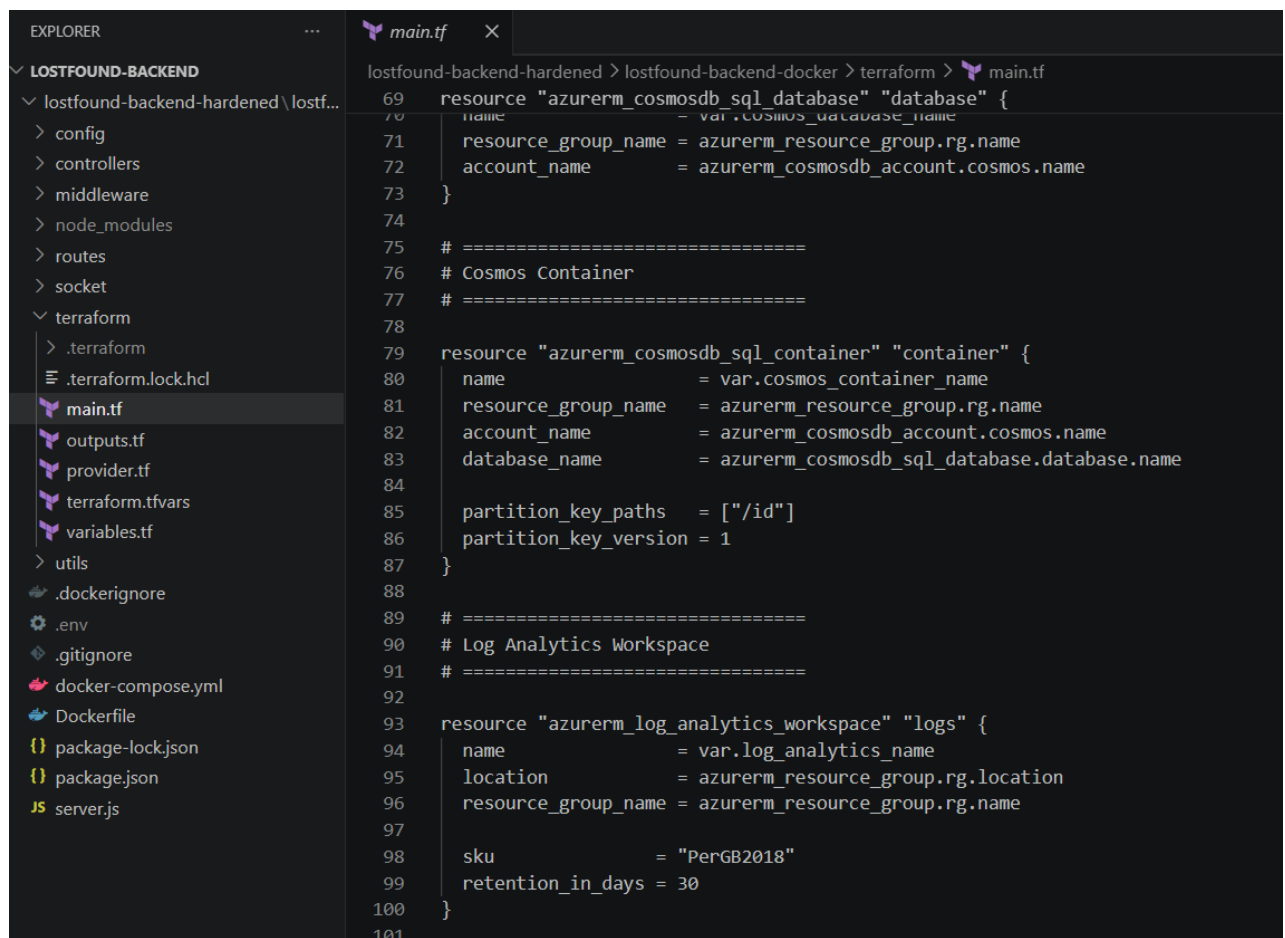
```
1 # =====
2 # Resource Group
3 # =====
4
5 resource "azurerm_resource_group" "rg" {
6   name     = var.resource_group_name
7   location = var.location
8 }
9
10 # =====
11 # Storage Account
12 # =====
13
14 resource "azurerm_storage_account" "storage" {
15   name                 = var.storage_account_name
16   resource_group_name = azurerm_resource_group.rg.name
17   location              = azurerm_resource_group.rg.location
18
19   account_tier                = "Standard"
20   account_replication_type    = "LRS"
21
22   min_tls_version            = "TLS1_2"
23
24   allow_nested_items_to_be_public = false
25 }
26
27 # =====
28 # Blob Container
29 # =====
30
31 resource "azurerm_storage_container" "uploads" {
32   name                  = var.blob_container_name
33   storage_account_id   = azurerm_storage_account.storage.id
```

Figure 8. Main.tf code



```
38 # Cosmos DB Account
39 # =====
40
41 resource "azurerm_cosmosdb_account" "cosmos" {
42   name                 = var.cosmos_account_name
43   location              = azurerm_resource_group.rg.location
44   resource_group_name = azurerm_resource_group.rg.name
45
46   offer_type = "Standard"
47   kind       = "GlobalDocumentDB"
48
49   consistency_policy {
50     consistency_level = "Session"
51   }
52
53   geo_location {
54     location          = azurerm_resource_group.rg.location
55     failover_priority = 0
56   }
57
58   capabilities {
59     name = "EnableServerless"
60   }
61
62   free_tier_enabled = true
63 }
64
65 # =====
66 # Cosmos Database
67 # =====
68
69 resource "azurerm_cosmosdb_sql_database" "database" {
70   name = var.cosmos_database_name
```

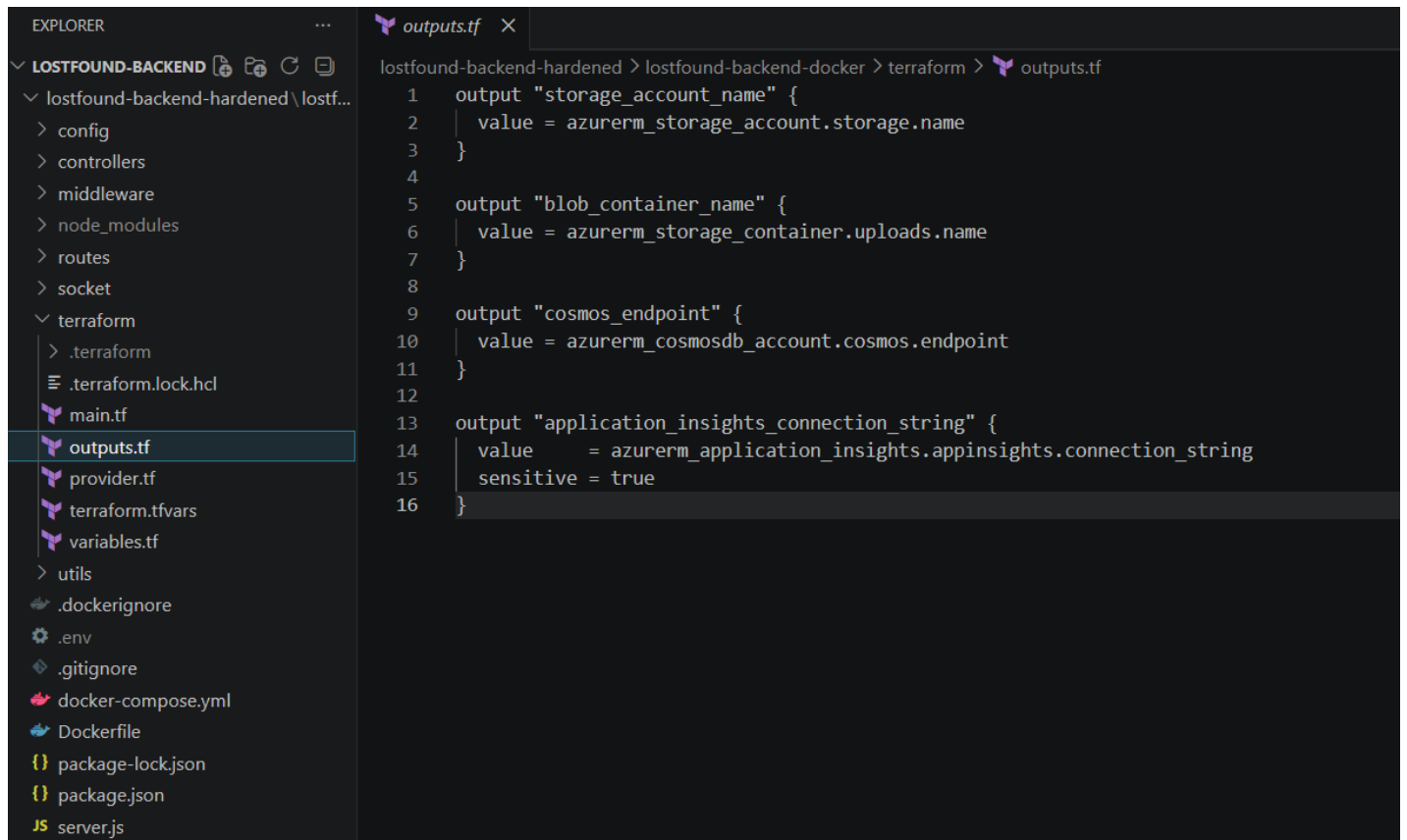
Figure 9. Main.tf code



```
69 resource "azurerm_cosmosdb_sql_database" "database" {
70   name                = var.cosmos_database_name
71   resource_group_name = azurerm_resource_group.rg.name
72   account_name        = azurerm_cosmosdb_account.cosmos.name
73 }
74
75 # =====
76 # Cosmos Container
77 # =====
78
79 resource "azurerm_cosmosdb_sql_container" "container" {
80   name                = var.cosmos_container_name
81   resource_group_name = azurerm_resource_group.rg.name
82   account_name        = azurerm_cosmosdb_account.cosmos.name
83   database_name        = azurerm_cosmosdb_sql_database.database.name
84
85   partition_key_paths = ["/id"]
86   partition_key_version = 1
87 }
88
89 # =====
90 # Log Analytics Workspace
91 # =====
92
93 resource "azurerm_log_analytics_workspace" "logs" {
94   name                = var.log_analytics_name
95   location             = azurerm_resource_group.rg.location
96   resource_group_name = azurerm_resource_group.rg.name
97
98   sku                = "PerGB2018"
99   retention_in_days = 30
100 }
101
```

Figure 10. Main.tf code

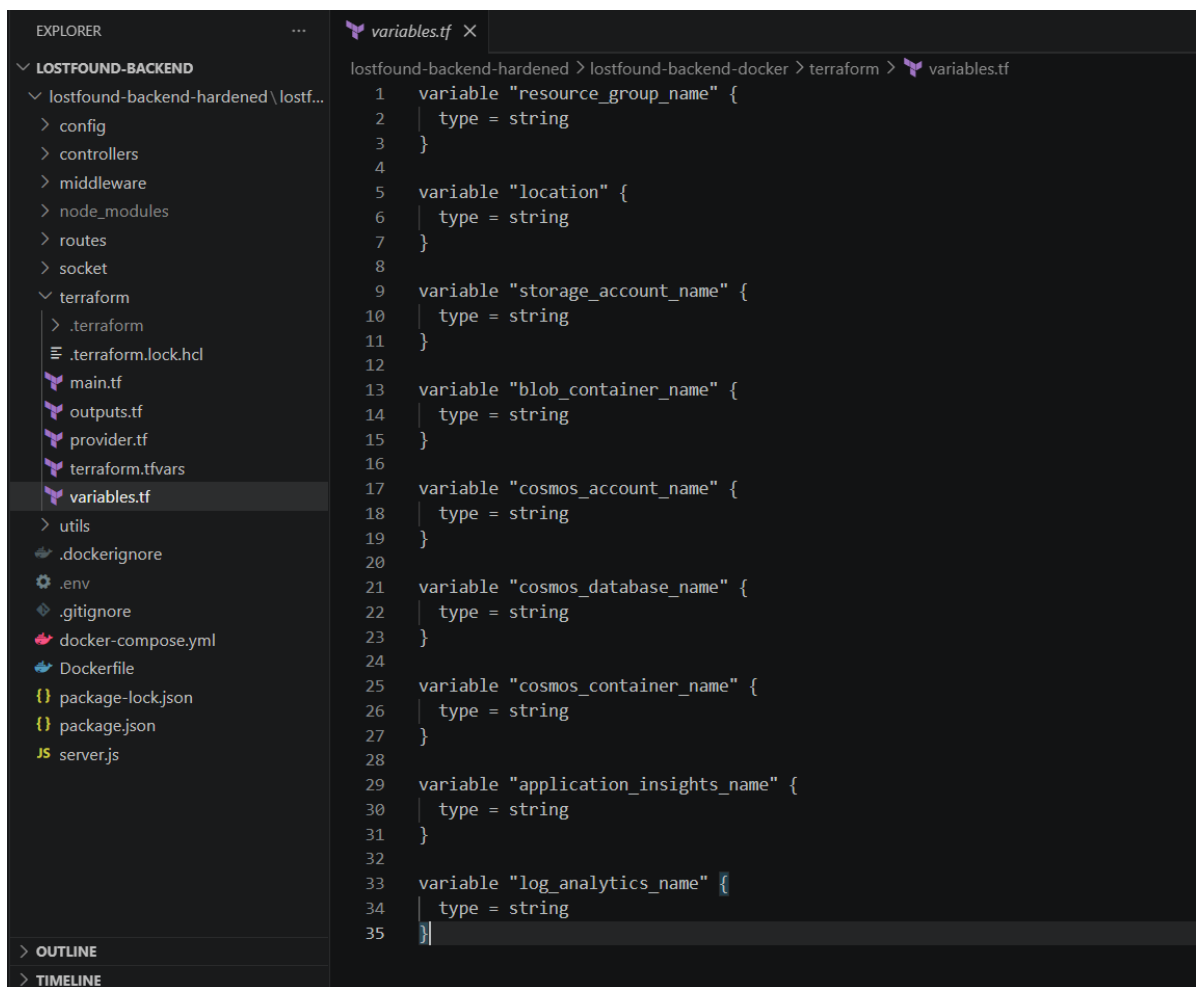
[outputs.tf](#)



```
1  output "storage_account_name" {
2    value = azurerm_storage_account.storage.name
3  }
4
5  output "blob_container_name" {
6    value = azurerm_storage_container.uploads.name
7  }
8
9  output "cosmos_endpoint" {
10   value = azurerm_cosmosdb_account.cosmos.endpoint
11 }
12
13 output "application_insights_connection_string" {
14   value      = azurerm_application_insights.appinsights.connection_string
15   sensitive = true
16 }
```

Figure 11. Outputs.tf code

[Variables.tf](#)



```
1  variable "resource_group_name" {
2    | type = string
3  }
4
5  variable "location" {
6    | type = string
7  }
8
9  variable "storage_account_name" {
10   | type = string
11 }
12
13 variable "blob_container_name" {
14   | type = string
15 }
16
17 variable "cosmos_account_name" {
18   | type = string
19 }
20
21 variable "cosmos_database_name" {
22   | type = string
23 }
24
25 variable "cosmos_container_name" {
26   | type = string
27 }
28
29 variable "application_insights_name" {
30   | type = string
31 }
32
33 variable "log_analytics_name" {
34   | type = string
35 }
```

Figure 12. *variables.tf* code

Integrated AI services:

1. Azure content moderation

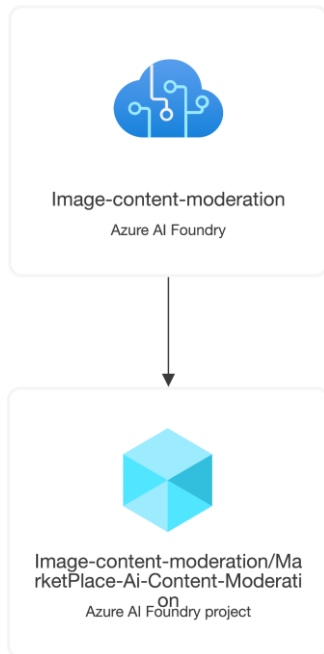


Figure 13. Azure content moderation

2. Azure Container Apps



Figure 14. Description Generation with gemini vision pro

Deployment resource visualizer

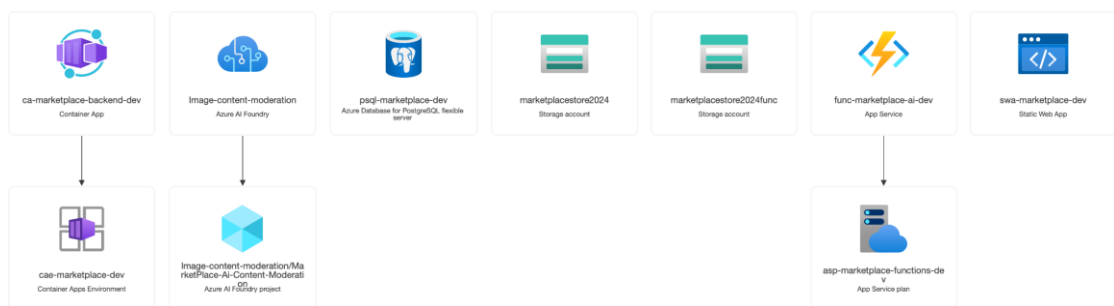


Figure 15. Network Visualizer